



Module: Développer en back-end

11. API en Laravel



Filière: Développement digital – option web full stack

Cours réalisé par:

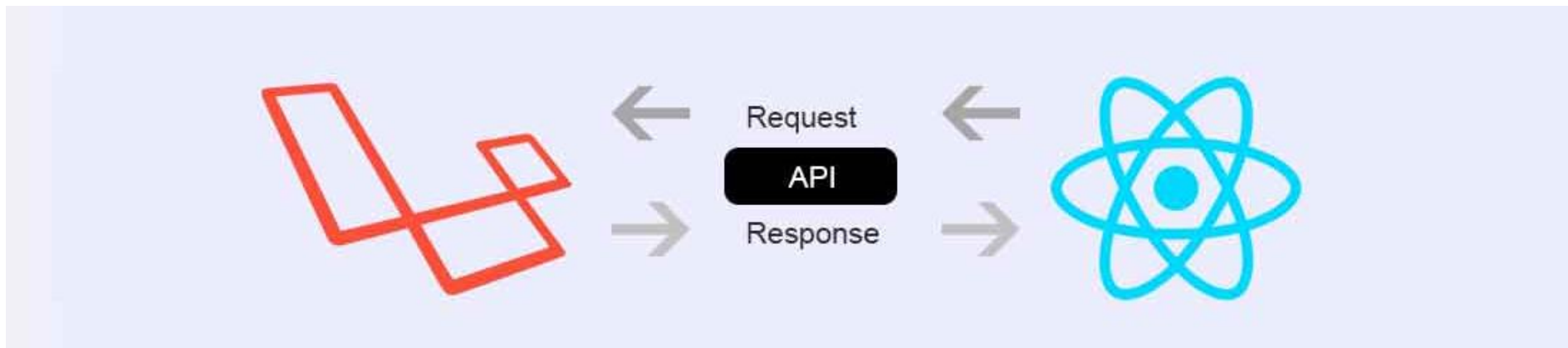
- Asmae YOUALA (ISTA ADARISSA- Fès)

PLAN DU COURS:

1. Introduction aux APIs en Laravel
2. Configuration des fichiers (Modèle, Contrôleur de ressource ...)
3. Configuration des routes
4. Outil de test d'API
5. Exemple de manipulation d'une ressource (affichage, ajout, modification et suppression)
6. Exemple d'utilisation en React avec *createAsyncThunk*

1. Introduction aux APIs en Laravel

- Pour les applications web, une **API** (Application Programming Interface) ou « Interface de Programmation d'Applications » en français, est un ensemble de définitions et de protocoles qui permettent à des applications **de communiquer et de s'échanger mutuellement des données ou des services**.
- **Laravel** peut également servir de **backend API** pour une application JavaScript monopage ou une application mobile.
- Par exemple, on peut utiliser Laravel comme backend API pour une application React JS. Dans ce contexte, on peut utiliser Laravel pour assurer l'authentification et le stockage/récupération de données pour notre application, tout en profitant des puissants services de Laravel tels que les files d'attente, les e-mails, les notifications, etc.



2. Configuration des fichiers

- Configuration de la base de données, du modèle et du contrôleur de ressource:

▪ `php artisan make:model Stagiaire -crms -api`

- Cette commande permet de créer un modèle **Stagiaire**, un contrôleur de ressource **StagiaireController**, un fichier de migration et un fichier seeder.
- L'option `--api` permet de retirer les méthodes **create** et **edit** du contrôleur puisqu'elles ne sont pas utiles pour une API. Il va rester **index**, **store**, **show**, **update** et **destroy**.

Ainsi, le code généré dans le contrôleur sous `app/Http/Controllers/StagiaireController.php` est présenté ainsi:

```
app > Http > Controllers > StagiaireController.php > ...
3  namespace App\Http\Controllers;
4
5  use App\Models\Stagiaire;
6  use Illuminate\Http\Request;
7
8  class StagiaireController extends Controller
9  {
10     /**
11      * Display a listing of the resource.
12      */
13     public function index() { }
14     /**
15      * Store a newly created resource in storage.
16      */
17     public function store(Request $request) { }
18     /**
19      * Display the specified resource.
20      */
21     public function show(Stagiaire $stagiaire) { }
22     /**
23      * Update the specified resource in storage.
24      */
25     public function update(Request $request, Stagiaire $stagiaire) { }
26     /**
27      * Remove the specified resource from storage.
28      */
29     public function destroy(Stagiaire $stagiaire) { }
30 }
31
```

3. Configuration des routes

- Installation de l'API dans le cas de Laravel 11:

- Le skeleton, par défaut d'un projet Laravel 11, ne prend pas en charge les APIs, pour ce, on va l'installer avec la commande suivante:

```
php artisan install:api
```

- Configuration des routes:

Pour définir les routes « stagiaire.* » (stagiaire.index, stagiaire.show, stagiaire.store, ...) de l'API, appelons la méthode **Route::apiResource()** au niveau du fichier **/routes/api.php** :

- `Route::apiResource("stagiaire", StagiaireController::class);`
- Pour afficher la liste des routes API générées, utilisons la commande: **php artisan route:list --path=api**

```
asmae@DESKTOP-PGQ5OJJ MINGW64 ~/Desktop/laravel/laravel_react/back-end (main)
$ php artisan route:list --path=api
```

```
GET|HEAD      api/stagiaire ..... stagiaire.index > StagiaireController@index
POST          api/stagiaire ..... stagiaire.store > StagiaireController@store
GET|HEAD      api/stagiaire/{stagiaire} ..... stagiaire.show > StagiaireController@show
PUT|PATCH    api/stagiaire/{stagiaire} ..... stagiaire.update > StagiaireController@update
DELETE        api/stagiaire/{stagiaire} ..... stagiaire.destroy > StagiaireController@destroy
```

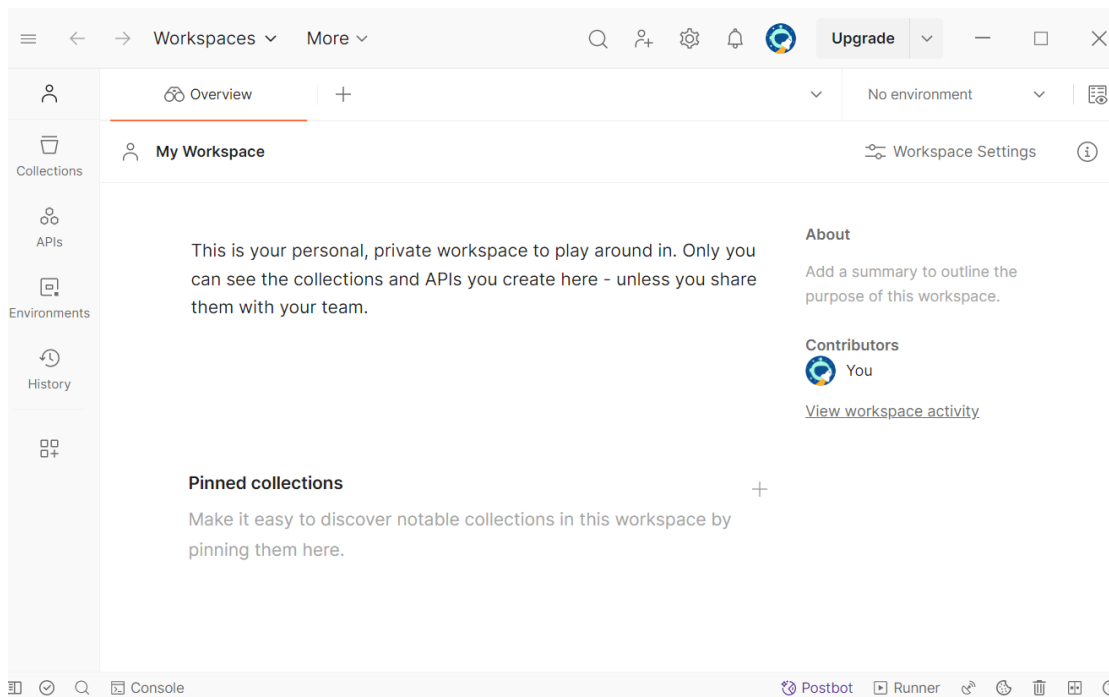
4. Outil de test d'API

-Tester les points de termination (end-point) d'un API :

Lorsque nous définissons des routes avec les verbes HTTP (GET, POST, PUT/PATCH, DELETE ...), il n'est pas possible de les tester tous via le navigateur, comme pour une simple requête GET.

Nous allons utiliser un logiciel appelé **Postman** qui permet d'**envoyer toutes sortes de requêtes et de les personnaliser très finement**. Il permet également de gérer l'authentification, les scripts de tests, etc.

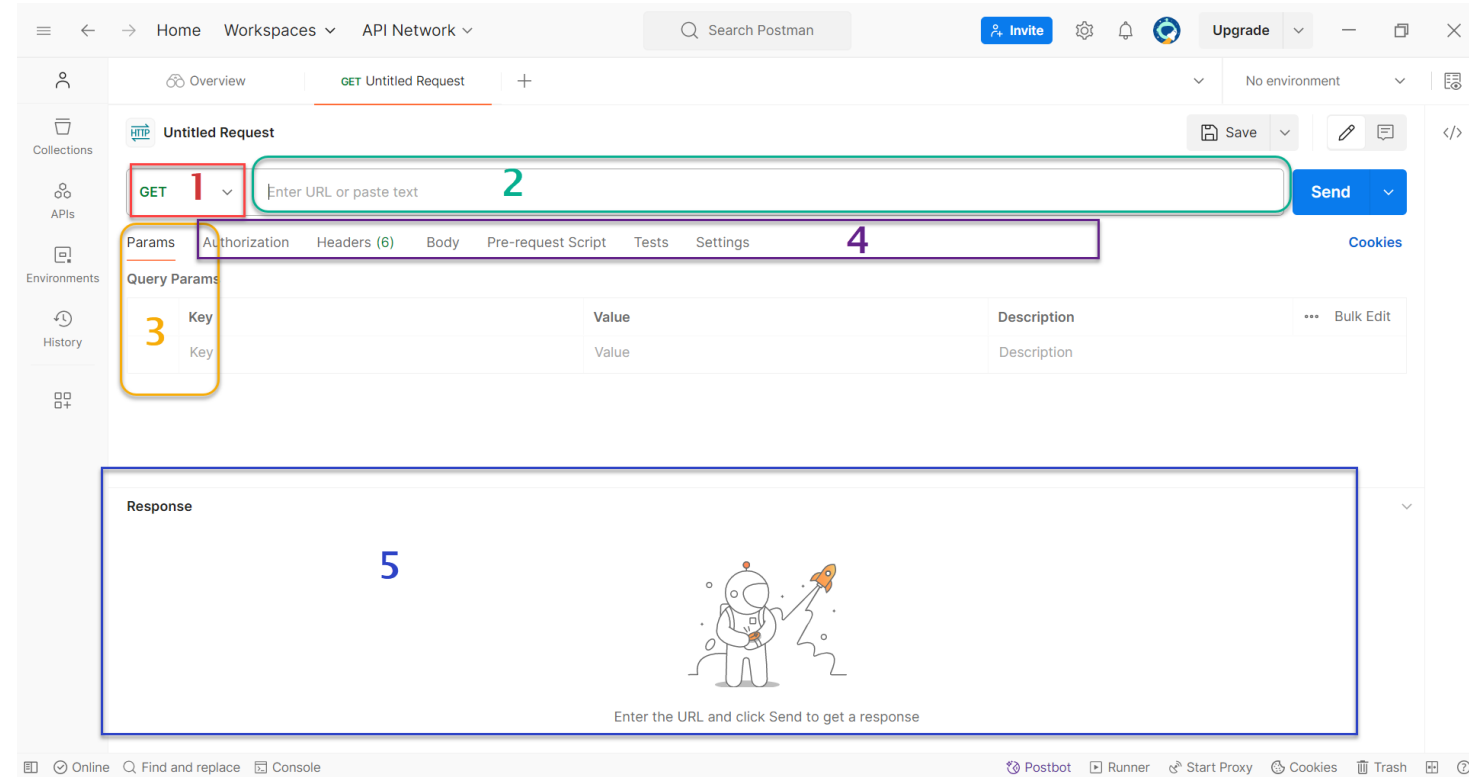
Lien de téléchargement de Postman: <https://www.postman.com/downloads/>



4. Outil de test d'API

-Description de l'interface principale de Postman:

- Voici les principales fonctionnalités à connaître pour nos tests :
- (1) : Ici vous pouvez choisir le type de requête à envoyer : GET, POST, PUT, DELETE, etc.
- (2) : L'URL de l'API.
- (3) : Les paramètres à passer avec l'URL. Dans le cas où vous avez des paramètres nommés, comme ?prenom=alaoui, vous allez pouvoir ajouter dans le formulaire comme paramètre *prenom*, et comme valeur, *alaoui*. Ceci se révèle particulièrement utile quand vous avez beaucoup de paramètres qu'il faut organiser.
- (4) : Ici vous trouverez tout ce qui constitue une requête HTTP (header, body, etc.). Vous pouvez à partir de là créer votre requête de façon totalement personnalisée.
- (5) : Après le clic sur le bouton "send", c'est ici que vous aurez le résultat de la requête



5. Exemple de manipulation d'une ressource

a. Affichage de tous les stagiaires:

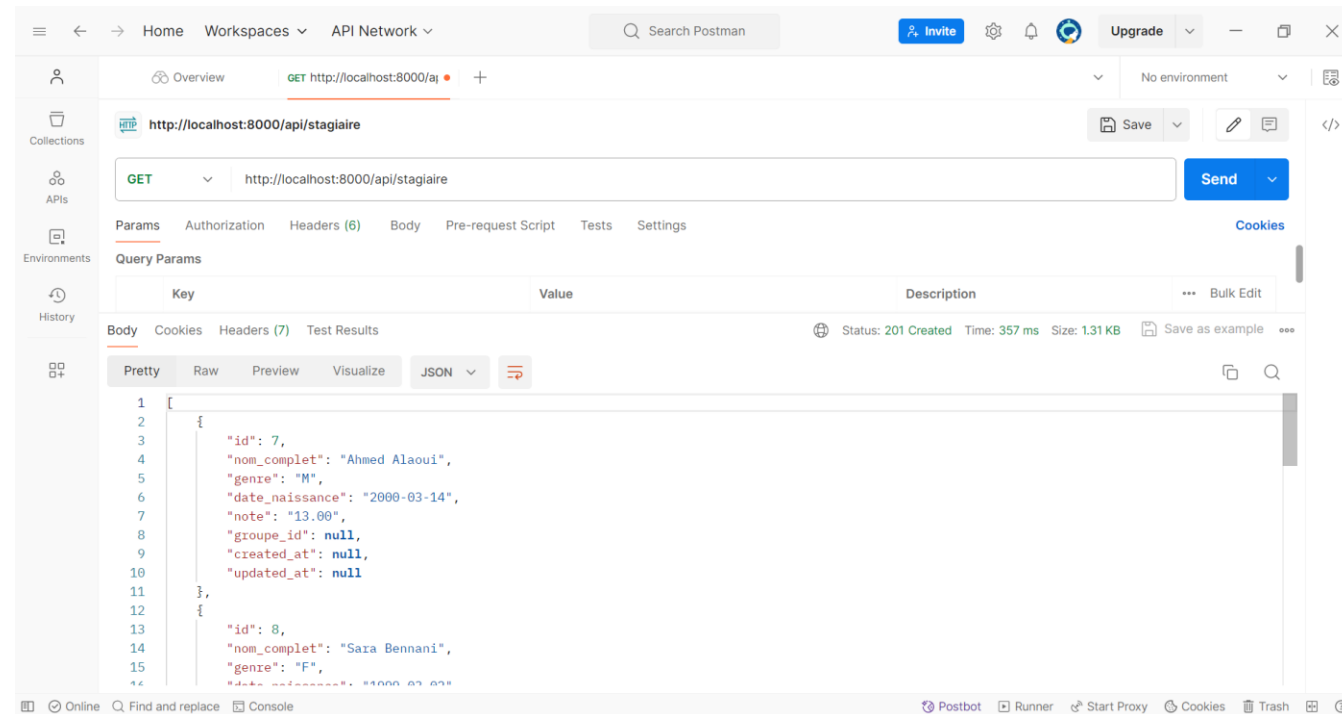
Méthode	URI	Name	Action
GET HEAD	api/stagiaire	stagiaire.index	App\Http\Controllers\API\StagiaireController@index

La méthode **index()** de la route nommée « **stagiaire.index** » retournera la liste des stagiaires sous format json :

```
public function index()    {  
    $stagiaires= Stagiaire::all();  
    return response()->json($stagiaires,201);  
}
```

La méthode **response()->json(\$data, \$code)** :

- **\$data** représente le body (corps)
- **\$code** le code de statut de la réponse HTTP, retourne une réponse avec le header « Content-Type : applicaton/json ». Nous obtenons le code de statut de réponse « **200 OK** » lorsqu'une requête réussit et « **404 Not Found** » lorsque la ressource demandée n'est pas trouvée.



5. Exemple de manipulation d'une ressource

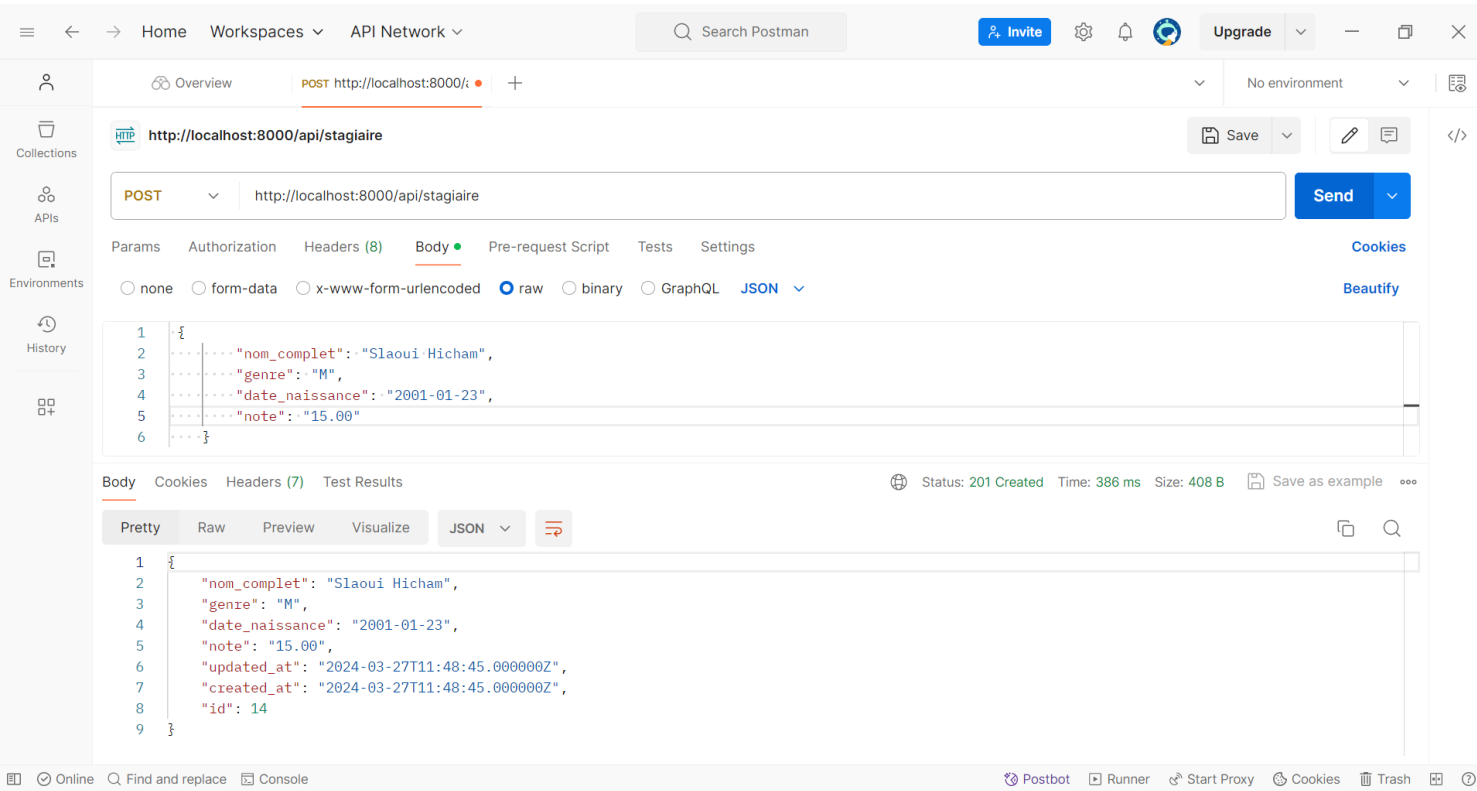
b. Ajout d'un nouveau stagiaire :

Méthode	URI	Name	Action
POST	api/stagiaire/	stagiaire.store	App\Http\Controllers\API\StagiaireController@store

La méthode **store()** de la route nommée « **stagiaire.store** » enregistrera un nouveau stagiaires :

```
public function store(Request $request) {  
    $nv_stagiaire= Stagiaire::create(  
        $request->all());  
    return response()->json($nv_stagiaire,201);  
}
```

On insère le nouveau stagiaire dans la table « **stagiaires** » puis on retourne une réponse JSON avec les informations du \$nv_stagiaire nouvellement créé avec le code "201 Created" »



5. Exemple de manipulation d'une ressource

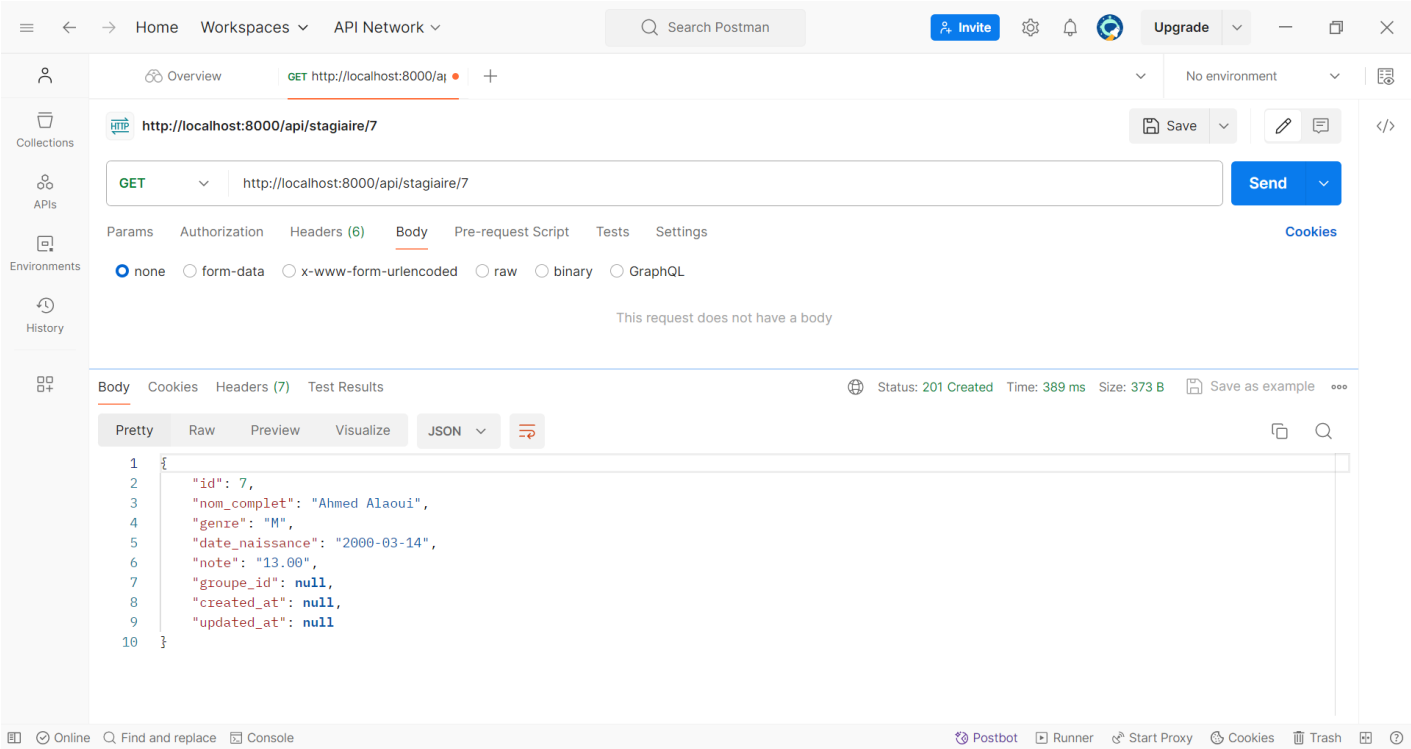
c. Affichage d'un stagiaire :

Méthode	URI	Name	Action
GET HEAD	api/stagiaire/{stagiaire}	stagiaire.show	App\Http\Controllers\API\StagiaireController@show

La méthode **show()** de la route nommée « `stagiaire.show` » affiche les informations du stagiaire dont l'id est passé en URL :

```
public function show(Stagiaire $stagiaire) {  
    return response()->json($stagiaire,201);  
}
```

Si le stagiaire est trouvé, on retourne ses informations avec le code 201 sinon, on retourne une réponse avec le code 404.



5. Exemple de manipulation d'une ressource

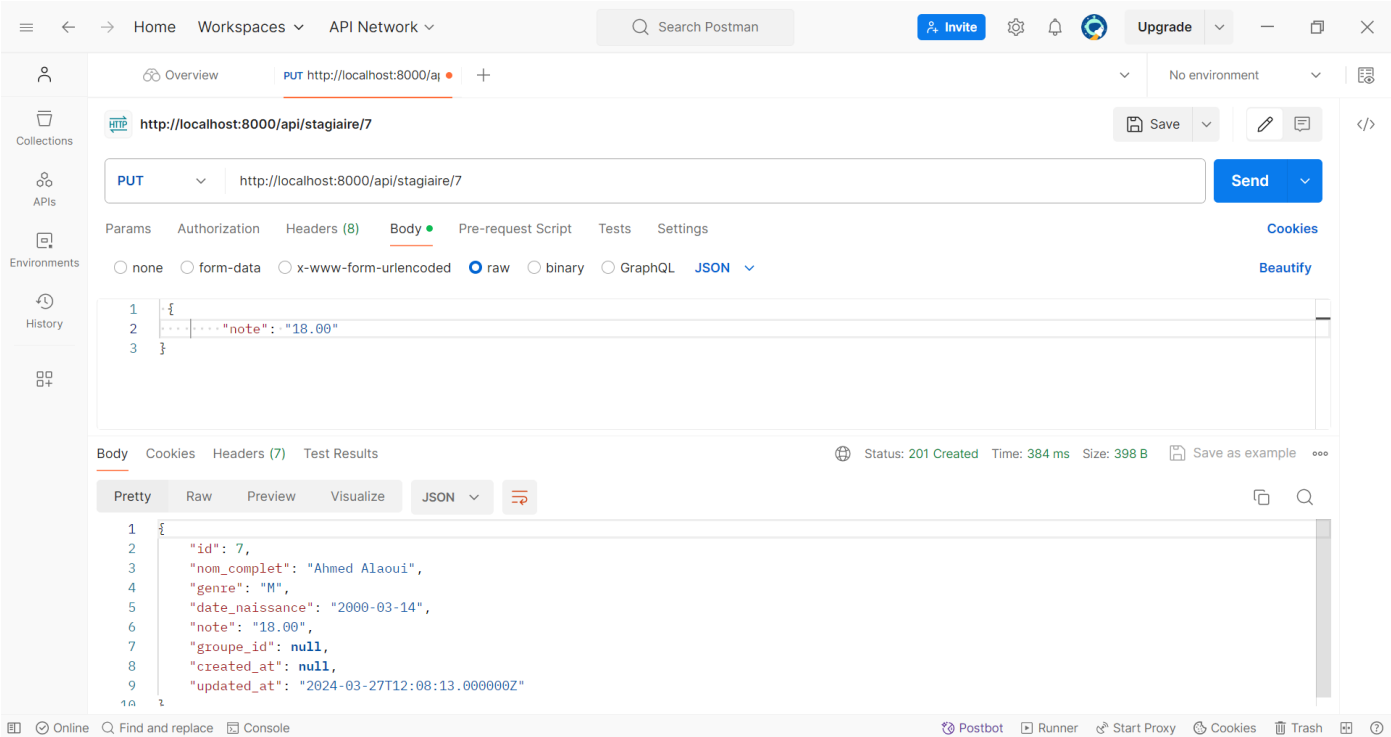
c. Modification d'un stagiaire :

Méthode	URI	Name	Action
PUT PATCH	api/stagiaire/{stagiaire}	stagiaire.update	App\Http\Controllers\API\StagiaireController@update

La méthode **update()** de la route nommée « stagiaire.update permet de mettre à jour un stagiaire spécifié :

```
public function update(Request $request,
    Stagiaire $stagiaire) {
    $stagiaire ->update($request->all());
    return response()->json($stagiaire,201);
}
```

On modifie les informations de du stagiaire \$stagiaire puis on retourne les nouvelles informations du stagiaire modifié via une réponse JSON.



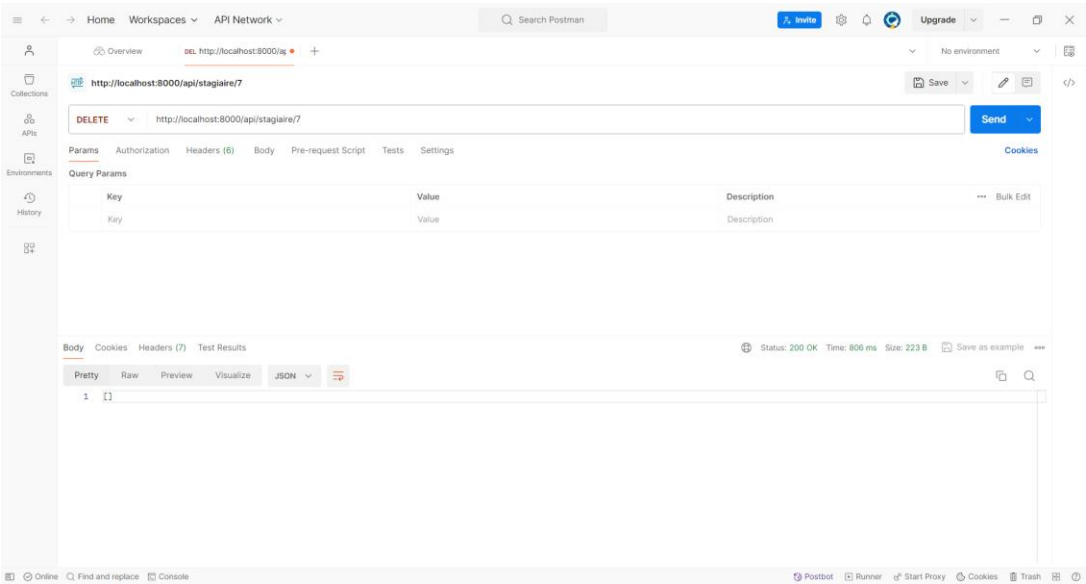
5. Exemple de manipulation d'une ressource

c. Suppression d'un stagiaire :

Méthode	URI	Name	Action
DELETE	api/stagiaire/{stagiaire}	stagiaire.destroy	App\Http\Controllers\API\StagiaireController@destroy

La méthode **destroy()** de la route nommée "stagiaire.destroy" permet de supprimer le stagiaire spécifié :

```
public function destroy(Stagiaire $stagiaire) {  
    if ($stagiaire==null){  
        return response()->json(['message'=>'Le  
stagiaire est introuvable'],404);  
    }  
    $stagiaire->delete();  
    // On retourne la réponse JSON  
    return response()->json();  
}
```



6. Exemple d'utilisation en React avec *createAsyncThunk* :

```
import { createAsyncThunk } from "@reduxjs/toolkit";
import axios from "axios";

const getStagiaires = createAsyncThunk("stg/get", async
() => {
  return axios
    .get("http://127.0.0.1:8000/api/stagiaire")
    .then((response) => response.data);
});

const showStagiaire = createAsyncThunk("stg/show", async
(id) => {
  return axios
    .get(`http://127.0.0.1:8000/api/stagiaire/${id}`)
    .then((response) => response.data);
});

const createStagiaire = createAsyncThunk("stg/create",
async (nvstg) => {
  return axios
    .post("http://127.0.0.1:8000/api/stagiaire", nvstg)
    .then((response) => response.data);
});
```

```
const updateStagiaire = createAsyncThunk("stg/update",
async (stg) => {
  return axios
    .put(`http://127.0.0.1:8000/api/stagiaire/${stg.id}`
, stg)
    .then((response) => response.data);
});

const deleteStagiaire = createAsyncThunk("stg/delete",
async (id) => {
  return axios
    .delete(`http://127.0.0.1:8000/api/stagiaire/${id}`)
    .then((response) => response.data);
});

export {
  getStagiaires,
  createStagiaire,
  updateStagiaire,
  deleteStagiaire,
  showStagiaire,
};
```